

2026编译原理大作业项目报告

1 项目概述

1.1 作业需求说明

根据《2026编译实践大作业要求》：

- 实现扩展L26语言编译器，支持 `int` 整型、`bool` 布尔、`set` 集合三种数据类型；
- `set`集合元素上限200；
- 编译器输出标准类P-Code三字段指令，配套虚拟机解释运行；
- 语言语法包含块、变量声明、赋值、if-else、while循环、读写语句、集合专用语法；
- 集合功能：集合字面量 `{}`、`union` 并集、`inter` 交集、`add/remove` 增删、`in` 成员判断、`isempty` 空集判断；
- 支持多层嵌套作用域、变量遮蔽、完整静态语义检查；
- 界面展示P-Code清单、内存管理、单步运行（加分项）；
- 集合相等 `==` / 不等 `!=` 判定（加分项）。

1.2 开发方案选型

本项目选择**第三组 Coco/R C++实现**：

- Coco/R从ATG文法文件自动生成词法扫描器与LL(1)递归下降语法解析器；
- 在ATG文法中嵌入语义动作，同步完成符号表管理、类型检查、P-Code代码生成；
- 采用标准PL/0三字段P-Code格式，设计集合扩展OPR子码；
- 前端提供Web可视化界面，通过Node.js桥接调用C++后端。

1.3 整体架构



两层运行模式：

- **普通模式**: `l26compiler.exe <file>` 编译并可选执行虚拟机
- **JSON模式**: `l26compiler.exe <file> --json` 输出结构化JSON供前端使用
- **前端模式**: `node server.js` 启动HTTP桥接, 浏览器访问 `localhost:8765` 使用可视化界面

2 L26语言完整EBNF文法定义

2.1 EBNF元符号

- `::=` 定义; `|` 或; `{ }` 0次及以上重复; `[ ]` 可选; 双引号为终结符

2.2 完整EBNF文法

```

<program>      ::= <block>
<block>        ::= "{" <decls> <stmts> "}"
<decls>        ::= { <decl> }
<decl>         ::= <type> ID ";"
<type>         ::= "int" | "bool" | "set"

<stmts>        ::= { <stmt> }
<stmt>         ::= <assign_stmt> | <if_stmt> | <while_stmt>
                  | <io_stmt> | <block> | <set_op_stmt>

<assign_stmt>  ::= ID "=" <expr> ";"
<if_stmt>      ::= "if" "(" <bexpr> ")" <stmt> [ "else" <stmt> ]
<while_stmt>   ::= "while" "(" <bexpr> ")" <stmt>

<io_stmt>      ::= "write" <expr> ";" | "read" ID ";"
<set_op_stmt>  ::= "add" ID <aexpr> ";" | "remove" ID <aexpr> ";"

<expr>         ::= <bexpr> | <aexpr> | <set_expr>

/* 算术表达式 */
<aexpr>        ::= <aterm> { ("+" | "-") <aterm> }
<aterm>        ::= <afactor> { ("*" | "/" ) <afactor> }
<afactor>      ::= ID | NUM | "(" <aexpr> ")"

/* 逻辑表达式 */
<bexpr>        ::= <bterm> { "||" <bterm> }
<bterm>        ::= <bfactor> { "&&" <bfactor> }
<bfactor>      ::= "true" | "false" | "!" <bfactor>
                  | "(" <bexpr> ")" | <rel> | <set_test>

/* 关系运算 */
<rel>          ::= <aexpr> ("<" | "<=" | ">" | ">=" | "==" | "!=") <aexpr>

/* 集合表达式 */
<set_expr>     ::= "{" [ <aexpr> { "," <aexpr> } ] "}"
                  | ID "union" ID | ID "inter" ID

/* 集合测试 */
<set_test>     ::= <aexpr> "in" ID | "isempty" "(" ID ")"

```

2.3 关键字与运算符

- 关键字: `int`、`bool`、`set`、`if`、`else`、`while`、`read`、`write`、`add`、`remove`、`union`、`inter`、`in`、`isempty`、`true`、`false`
- 算术运算符: `+` `-` `*` `/`
- 关系运算符: `<` `<=` `>` `>=` `==` `!=`
- 逻辑运算符: `!` `&&` `||`
- 集合运算符: `union` `inter` `in`

3 项目架构与代码结构

3.1 完整目录树

```
L26Compiler-workbuddy/
|
├─ compiler/                                # 后端 (C++/Coco/R)
|   └─ L26.atg                             # Coco/R文法文件
|   └─ SymTab.h                           # 符号表 (栈式多级作用域)
|   └─ PCode.h                             # P-Code指令集 + 代码生成器
|   └─ VM.h                               # 类P-Code栈式虚拟机
|   └─ main.cpp                           # 程序入口
|   └─ Parser.frame                       # Coco/R生成模板
|   └─ Scanner.frame                     # Coco/R生成模板
|   └─ Scanner.h/cpp                     # [生成] 词法分析器
|   └─ Parser.h/cpp                      # [生成] 语法分析器
|   └─ 126compiler.exe                   # 编译产物
|
├─ Frontend/                             # 前端 (web可视化)
|   └─ index.html                       # 自包含前端页面 (JS VM + 单步调试)
|   └─ server.js                        # Node.js HTTP桥接服务器
|
├─ Test/                                 # 测试用例 (18个: 13正确 + 5错误处理)
|   └─ 正确功能测试
|   └─ test1.126                        # read输入 + in成员判断 + add集合添加 + while循环 +
write输出
|   └─ test2.126                        # 嵌套作用域 + 变量遮蔽+ write set
|   └─ test3.126                        # union并集 + inter交集 + remove删除 + isempty空集判断
+ 布尔与循环
|   └─ test4.126                        # 最小算术测试: 声明、赋值、加减乘除、括号
|   └─ test5.126                        # 最小布尔测试: true/false 赋值
|   └─ test6.126                        # 最小关系测试: int 变量大小比较 (>)
|   └─ test7.126                        # 集合字面量与 in: set 声明、{1,2,3} 构造、成员测试
|   └─ test8.126                        # 集合相等/不等判定: == 与 != 比较 (加分项)
|   └─ test_full.126                    # 全语法综合: 声明/赋
值/union/inter/in/isempty/if/write
|   └─ test_arithmetic.126             # 算术全覆盖: + - * /、一元负号、多层括号嵌套、运算符优先级
|   └─ test_control.126                 # 控制流全覆盖: if-else、嵌套if、while累加、&& || ! 布尔
组合
|   └─ test_io.126                     # I/O全覆盖: write int/bool/set、read、运算后write、集
合union后write
|   └─ test_set_advanced.126           # 集合高级操作: 空集{}、链式union/inter、add/remove循
环、in、isempty
```

		错误处理测试	—————
		test_error_undeclared.l26	# 未声明变量：赋值/运算/条件/write中使用未声明标识符（4处错误）
		test_error_duplicate.l26	# 重复声明：同类型及不同类型重复声明（2处错误）
		test_error_type.l26	# 类型不匹配： int=bool/bool=int/bool=set/set=bool/int=set（5处错误）
		test_error_set_ops.l26	# 集合操作参数类型错误： add/remove/in/union/inter/isempty（7处错误）
		test_error_set_relop.l26	# 集合非法关系运算符：< > <= >= 用于集合比较（4处错误）
		报告/	
		项目报告.md	

3.2 后端核心模块（Compiler/）

3.2.1 Coco/R文法文件（L26.atg）

L26.atg是整个编译器的核心，Coco/R据此自动生成Scanner和Parser。文件结构分四节：

CHARACTERS节：定义字符集（letter = A..Z a..z _，digit = 0..9）。

TOKENS节：定义Token识别规则。ident = letter {letter | digit} 识别标识符，number = digit {digit} 识别整数。关键字由Coco/R的KeywordMap在扫描阶段自动识别，无需在TOKENS逐一列举。

COMMENTS/IGNORE节：支持// 单行注释和/* */ 嵌套块注释，空白字符被忽略。

PRODUCTIONS节：每个产生式对应一个Parser方法，花括号(. . . .)内嵌C++语义动作，在语法匹配时同步执行。关键产生式及语义动作：

产生式	语义动作
Block	EnterScope() → 发射INT(占位) → Decls 统计变量数 → 回填INT.a = cnt+3 → Stmts → LeaveScope()
Decl	调用 Declare() 检查重复声明，分配偏移
AssignStmt	Lookup() 查变量 → Expr() 算类型 → 类型匹配检查 → EmitSTO() 存入
IfStmt	发射PC占位 → 解析then体 → 发射JMP占位 → PatchJump(JPC) → [解析else体] → PatchJump(JMP)
whileStmt	记录labelTop → 发射JPC占位 → 解析循环体 → 发射JMP回跳 → PatchJump(JPC)
Expr	用 IF(IsSetExprStart()) / IF(IsAExprStart()) 谓词消歧，后置 [...] 检测 relop/in/union/inter
SetExpr	计数元素 → EmitSetMake(cnt)；union/inter侧做set类型校验

3.2.2 符号表 (SymTab.h)

实现栈式多级作用域，纯头文件inline实现。

数据结构：

```
struct SymEntry {
    string name;      // 变量名
    string typeName;  // "int" / "bool" / "set"
    int level;        // 定义所在嵌套层（0为最外层）
    int offset;       // 当前块内偏移
};
```

符号表本体为 `vector<map<string, SymEntry>> scopes`，每一层作用域对应一个map。辅助变量 `curLevel` 记录当前嵌套深度，`offsetStack` 记录每层已分配偏移数。

核心方法：

- `EnterScope()`：`curLevel++`，压入空map和初始偏移0
- `LeaveScope()`：`curLevel--`，弹出栈顶map和偏移
- `Declare(name, typ)`：在`scopes[curLevel]`中插入，同层重名返回false
- `Lookup(name)`：从`curLevel`向下遍历`scopes[0]`，返回最近绑定，实现变量遮蔽
- `CurrentOffset()`：返回当前层已分配偏移数

偏移：变量偏移从0开始计数，代码生成时加3（`offset+3`），因为活动记录前3槽固定为SL/DL/RA联系单元。

3.2.3 P-Code代码生成器 (PCode.h)

包含三部分：

指令结构体：

```
struct PInstr {
    fct f; // 操作码（F_LIT/F_OPR/F_LOD/F_STO/F_CAL/F_INT/F_JMP/F_JPC）
    int l; // 层次差（静态链向上层数，无层次时为0）
    int a; // 操作数（常量值/变量偏移/跳转地址/OPR子码）
};
```

严格遵循三元组(f, l, a)格式。前缀 `F_` 为规避Windows SDK中CAL/INT宏冲突。

操作码枚举：8种基础操作码覆盖全部PL/0标准指令。

OPR子码枚举：标准子码（0-20）保持与PL/0一致，新增集合扩展子码（201-211）。

代码生成器类PCodeGen：

- `Emit(f, l, a)`：底层发射，返回指令序号
- 封装函数：`EmitLIT(val)`、`EmitLOD(level, off)`、`EmitSTO(level, off)`、`EmitINT(size)`、`EmitRET()`、`EmitJMP(addr)`、`EmitJPC(addr)`、`EmitBinOp(op)`、`EmitRelOp(op)`、`EmitBoolOp(kind)`、`EmitBoolNot()`、`EmitWRITE(typ)`、`EmitWRITELN()`、`EmitREAD()`

- 集合发射：EmitSetMake(cnt) (cnt存l字段)、EmitSetUnion()、EmitSetInter()、EmitSetIn()、EmitSetIsEmpty()、EmitSetAdd()、EmitSetRemove()、EmitSetEQ()、EmitSetNE()
- PatchJump(idx)：将idx处指令的a字段改为当前地址，解决if/while前向跳转
- PrintListing()：格式化输出全部P-Code汇编清单

关键设计：EmitSetMake(cnt) 将元素数存入l字段 (Emit(F_OPR, cnt, OPR_SET_MK))，a字段保留OPR子码值201，确保VM可通过 case OPR_SET_MK 正确匹配。

3.2.4 虚拟机 (VM.h)

寄存器：

- s[2048]：整型操作数栈
- p：程序计数器（当前指令序号）
- b：基址寄存器（当前活动记录起始地址）
- t：栈顶寄存器

静态链寻址：

```
int Base(int l, int cur_b) {
    while (l > 0) { cur_b = s[cur_b]; l--; }
    return cur_b;
}
```

沿 s[b]（即SL）向上跳l层，获取目标层基址。LOD/STO指令的偏移相对该基址计算。

活动记录布局：

```
s[b+0] = SL // 指向上一层活动记录的b
s[b+1] = DL // 调用方b（主程序为0）
s[b+2] = RA // 返回地址（主程序为0）
s[b+3..b+2+n] = n个局部变量
```

INT指令分配 a 个栈槽 (t += a)，其中 a = 局部变量数 + 3。RET指令销毁帧：t = b-1; p = s[b+2]; b = s[b+1]。

集合扩展存储：static set<int> g_setPool[2048]，栈中仅存集合池下标。VM通过 g_setPool[t] 访问集合内容，不破坏整型栈结构。集合200上限 MAX_SET_SIZE=200 在SET_MK（构造时检查）、SET_UNION（并集后检查）、SET_ADD（添加前检查）三处强制。

执行流程：

1. Run()：循环调用 ExecInstr() 直至halted或PC越界
2. Step()：单步执行一条指令
3. ExecInstr()：switch分发8种基础指令
4. ExecOPR()：switch分发30多种OPR子码，涵盖算术、关系、逻辑、IO、全部集合操作

3.2.5 主程序入口 (main.cpp)

支持两种运行模式：

普通模式 (`l26compiler.exe <file>`)：

- 1. `system("chcp 65001")` 设置UTF-8控制台
- 2. 创建Scanner读取源文件
- 3. 创建Parser执行语法/语义分析
- 4. 有错误则打印计数并退出
- 5. 无错误则打印P-Code清单，询问是否执行VM
- 6. 初始化VM加载P-Code，执行并输出结果

JSON模式 (`l26compiler.exe <file> --json`)：

- 1. 第一遍：单独创建Scanner，循环调用 `scan()` 收集全部Token（类型、值、行号、列号）
- 2. 第二遍：新建Scanner+Parser，设置 `errors->silent=true` 抑制 `wprintf` 输出，调用 `parse()` 收集P-Code
- 3. 收集错误：从 `errors->errorMsgs`（`vector<ErrorEntry>`）读取，`wchar_t`→UTF-8正确编码中文字符
- 4. P-Code附带估算行号：按指令序号比例映射到token流位置
- 5. 输出一行完整JSON（tokens数组 + pcode数组 + errors数组）到 `stdout`
- 6. 不执行VM，不交互询问

静态辅助函数：

- `ws2s(wchar_t*)`：宽字符转窄字符（仅用于ASCII token值）
- `wchar_to_utf8(wstring)`：UTF-16LE→UTF-8转换，正确编码中文错误信息
- `jsonEscape(string)`：JSON字符串转义
- `tokenKindName(int)`：38种token类型编号→字符串映射

3.3 前端模块 (Frontend/)

前端通过Node.js桥接调用C++后端，浏览器内实现可视化编译与单步调试。词法/语法分析完全由后端完成，前端只负责UI渲染和虚拟机执行，避免重复实现。

3.3.1 Node.js桥接服务器 (server.js)

技术方案：浏览器无法直接执行本地exe文件，需要HTTP中间层。server.js提供轻量级HTTP服务器，前端通过fetch POST源码到此服务器，服务器写临时文件后调用 `l26compiler.exe --json`，将JSON结果返回前端。

路由设计：

方法	路径	功能
GET	/	读取 <code>index.html</code> 并返回，浏览器直接访问 <code>localhost:8765</code> 即可使用
POST	/compile	接收 <code>{"source":"..."}</code> ，写临时文件→调用编译器→返回JSON

方法	路径	功能
OPTIONS	<code>/compile</code>	CORS预检, 返回允许的Method和Header

POST `/compile`处理流程:

1. 解析请求体JSON, 提取source字段
2. 在系统临时目录创建 `126_<timestamp>.126` 文件, 写入源码
3. 调用 `execSync("126compiler.exe <tmpFile> --json")`, 超时10秒
4. 成功: 返回 200 + JSON (tokens + pcode + errors)
5. 失败: 返回 500 + 错误JSON
6. 删除临时文件

关键设计: server.js放在Frontend/目录下, 编译器路径为 `../Compiler/126compiler.exe`, 保持前后端目录分离。

3.3.2 可视化前端页面 (index.html)

结构分三部分: CSS样式、HTML布局、JavaScript逻辑。

界面布局:

控制栏 (编译+执行/仅编译/单步/示例/清空/状态)		
源代码	程序输出	内存栈(Frames)
P-Code指令序列(50%)		Token列表(50%)

F码和OPR码对齐: 前端JS定义了与后端PCode.h完全一致的枚举:

- `F = {LIT:0, OPR:1, LOD:2, STO:3, CAL:4, INT:5, JMP:6, JPC:7}`
- OPR 对象包含全部标准子码和集合扩展子码, 数值与后端 `oprCode` 枚举一一对应

JS虚拟机:

- 数据结构: `s[2048]` 整型栈、`setPool[]` 集合池、`p/b/t` 寄存器, 与后端VM.h完全对齐
- `base(l, b)`: 静态链寻址, 沿 `s[b]` 向上l层
- `stepAsync()`: 单步执行一条指令, 返回true表示需要read输入
- `execOPR(ins)`: 分发全部OPR子码, 逻辑与后端VM.h一致
- `pushSet(set) / popSet()`: 集合辅助存储, `setPool[t]` 存Set对象
- `setStr(s)`: 集合格式化输出 (升序排列)

编译流程 (doCompile函数):

1. `fetch(BACKEND, {method:'POST', body:JSON.stringify({source})})` 发往后端
2. 解析JSON响应, 提取tokens、pcode、errors
3. 先渲染Token表和P-Code表
4. 如有错误: 显示在输出区, 设置 `compileHadErrors=true`, 保存到 `savedErrors`

5. 返回 `compiledCode.length > 0`

单步调试流程：

1. 点击"开始单步"→ `startStep()` →编译→初始化VM→清空输出区→显示"有编译错误，可单步查看"提示
2. 点击"单步执行"→ `stepOnce()` → `vm.stepAsync()` 执行一条指令→高亮当前PC行→更新内存栈帧→更新输出
3. 每步检查当前P-Code指令行号是否匹配编译错误行号，命中则在输出区显示错误（同一错误只显示一次）
4. read语句触发 `promptInput()` 弹窗，用户输入后继续执行
5. `vm.halted`时自动禁用单步按钮

渲染函数：

函数	功能
<code>renderTokens(tokens)</code>	遍历JSON token数组，按关键字/标识符/数字/分隔符/运算符分类着色
<code>renderPCode(code)</code>	遍历P-Code数组，显示地址/F名称(l a)/备注
<code>highlightPCode(pc)</code>	高亮指定PC行并滚动到可见区域
<code>renderMemory(vm)</code>	沿静态链向上遍历所有帧，从外到内渲染每帧卡片，显示SL/DL/RA联系单元和全部局部变量，集合变量展开为 <code>{1,2,3}</code> ，当前帧紫色高亮
<code>renderOutput(output)</code>	展示VM运行输出
<code>updateStepInfo(vm)</code>	更新PC/b/t寄存器值，显示栈顶最近6个元素

错误展示策略：

- 编译+执行模式：错误直接显示在输出区，阻止执行
- 单步模式：编译时不清空输出区，只显示淡黄提示；单步到对应行时展示错误；`shownErrors` Set 确保每条错误只显示一次
- 后端返回的UTF-8中文错误消息正确渲染

输入弹窗： read语句触发Promise-based弹窗，用户输入整数后VM推送该值并继续执行。支持Enter键快速确认。

4 类P-Code规范与指令集设计

4.1 统一三元组格式

所有指令固定格式：（地址）助记符 l a

4.2 八种基础指令

指令	格式	语义
LIT	LIT 0 a	常数a压栈

指令	格式	语义
OPR	OPR 0 x	执行OPR子码x
LOD	LOD l a	加载base(l,b)+a处变量压栈
STO	STO l a	弹出栈顶存入base(l,b)+a
CAL	CAL l a	调用过程（L26暂不使用）
INT	INT 0 a	栈顶指针上移a（a=局部变量数+3）
JMP	JMP 0 a	无条件跳转
JPC	JPC 0 a	弹出栈顶，为0则跳转

4.3 OPR子码（与PL/0标准兼容 + 集合扩展）

标准子码（0-20）：

RET(0) / NEG(1) / ADD(2) / SUB(3) / MUL(4) / DIV(5) / ODD(6) /
EQ(8) / NE(9) / LT(10) / GE(11) / GT(12) / LE(13) /
WRITE(14) / WRITELN(15) / READ(16) /
AND(18) / OR(19) / NOT(20)

集合扩展子码（201-211）：

SET_MK(201): 弹cnt个整数构造集合（cnt存l字段）
SET_UNION(202): 弹两集合求并集
SET_INTER(203): 弹两集合求交集
SET_IN(204): 弹整数和集合测成员
SET_ISEMPY(205): 测栈顶集合是否空
SET_ADD(206): 弹整数加入栈顶集合
SET_REMOVE(207): 弹整数从栈顶集合删除
SET_EQ(208): 测试两集合相等
WRITE_BOOL(209): 输出布尔值
WRITE_SET(210): 输出集合
SET_NE(211): 测试两集合不等

4.4 虚拟机栈帧布局

```
s[b+0] = SL（Static Link，外层基址）  
s[b+1] = DL（Dynamic Link，调用方基址）  
s[b+2] = RA（Return Address，返回地址）  
s[b+3] = 局部变量0  
s[b+4] = 局部变量1  
...
```

Base(l,b)函数沿静态链向上l层寻址。

5 编译器完整运行流程

5.1 后端命令行模式

```
cd Compiler
g++ -std=c++11 -o l26compiler.exe main.cpp Parser.cpp Scanner.cpp      # 编译
l26compiler.exe ..\Tests\test_full.l26                                # 运行
# 查看P-Code清单，输入y执行虚拟机
```

5.2 前端可视化模式

```
cd Frontend
node server.js                # 启动桥接服务器
# 浏览器打开 http://localhost:8765
```

点击"编译+执行"全速运行；点击"开始单步"逐步调试，观察内存栈变化。

5.3 编译全流程

1. 词法分析：Scanner读取源文件，过滤空格/注释，输出Token流
2. 语法分析：Parser递归下降匹配EBNF，错误计数
3. 语义分析（同步执行）：符号表作用域管理、类型检查、集合运算合法性校验
4. 代码生成：调用PCodeGen发射P-Code，if/while先预留跳转后回填地址
5. 输出P-Code清单 / JSON输出
6. 虚拟机解释执行（可选）

5.4 跳转地址回填

if语句：解析条件 → JPC 0 -1(暂占) → 解析then体 → JMP 0 -1(暂占) → PatchJump(labelElse) → [解析else体] → PatchJump(labelEnd)

while语句：记录labelTop → 解析条件 → JPC 0 labelEnd(暂占) → 解析循环体 → JMP 0 labelTop → PatchJump(labelEnd)

6 测试用例与运行结果

共18个测试用例，分为正确功能测试（13个）和错误处理测试（5个），覆盖L26语言全部语法特性及全部语义错误类型。

6.1 正确功能测试

6.1.1 示例测试

test1.l26 — read 输入、in 成员判断、add 集合添加、if 条件、while 循环、write 输出

```
{
    int val;
    set s;
    s = {1, 2, 3};
```

```
read val;
if (val in s) {
    add s 10;
    write val;
}
while (val > 0) {
    val = val - 1;
    write val;
}
write 999;
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

🔍 仅编译

▶▶ 开始单步

↩ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 6 行输出

源代码 (L26)

程序输出

内存栈 (Stack Frames)
(单步执行时显示各帧变量)

P-Code 指令序列

Token 列表 (词法分析)

地址 操作码 操作数 备注

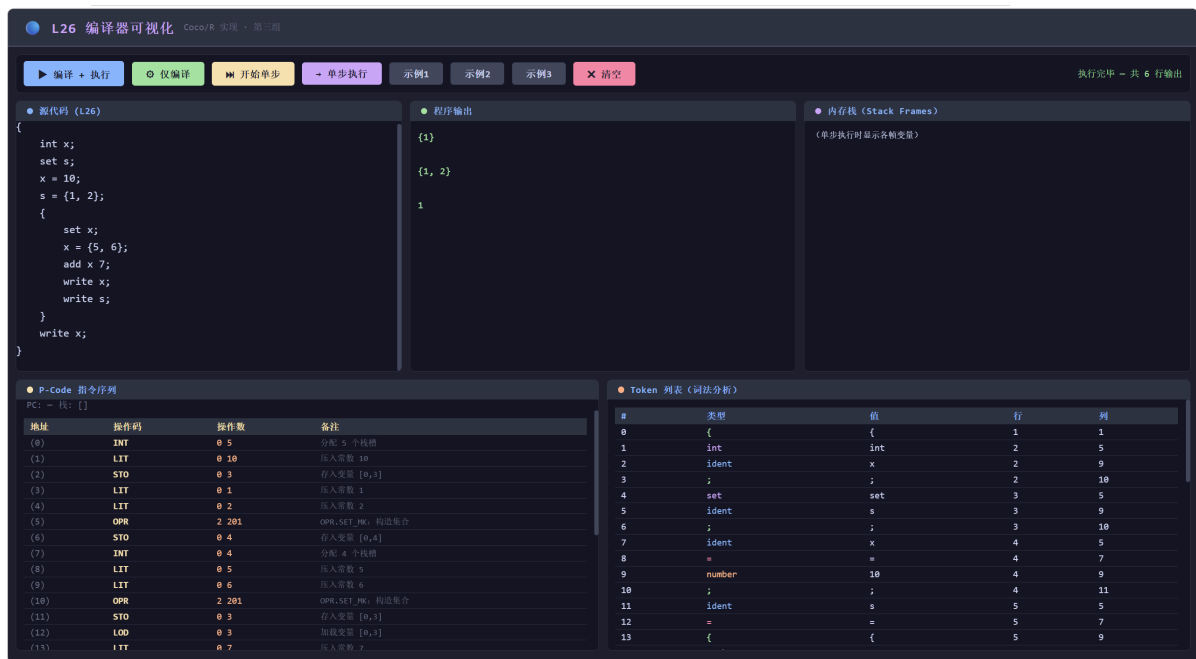
(0)	INT	0 5	分配 5 个栈槽
(1)	LIT	0 1	压入常数 1
(2)	LIT	0 2	压入常数 2
(3)	LIT	0 3	压入常数 3
(4)	OPR	3 201	OPR.SET_HRC: 构造集合
(5)	STO	0 4	存入变量 [0,4]
(6)	OPR	0 16	OPR.READ: 读输入
(7)	STO	0 3	存入变量 [0,3]
(8)	LDD	0 3	加载变量 [0,3]
(9)	LDD	0 4	加载变量 [0,4]
(10)	OPR	0 204	OPR.SET_HRC: 构造测试
(11)	JPC	0 21	跳转到行 = 21
(12)	INT	0 3	分配 3 个栈槽
(13)	LDD	1 4	加载变量 [1,4]

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	val	2	9
3	;	;	2	12
4	set	set	3	5
5	ident	s	3	9
6	;	;	3	10
7	ident	s	4	5
8	=	=	4	7
9	{	{	4	9
10	number	1	4	10
11	,	,	4	11
12	number	2	4	13
13	,	,	4	14

test2.l26 — 嵌套作用域、变量遮蔽（内层 set x 遮蔽外层 int x）、add、write set

```
{
    int x;
    set s;
    x = 10;
    s = {1, 2};
    {
        set x;
        x = {5, 6};
        add x 7;
        write x;
        write s;
    }
    write x;
}
```

运行结果：



6.1.2 基本语法测试

test3.l26 — union 并集、inter 交集、remove 删除、isempty 空集判断、布尔表达式、关系运算

```
{
  int a;
  int b;
  set s1;
  set s2;
  set s3;
  bool flag;

  a = 5;
  b = 3;
  s1 = {1, 2, 3, 4, 5};
  s2 = {3, 4, 5, 6, 7};

  s3 = s1 union s2;
  write s3;

  s3 = s1 inter s2;
  write s3;

  remove s1 3;
  write s1;

  flag = isempty(s1);
  write flag;

  if ((a > b) && (a != 0)) {
    write a;
  }
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

🔍 仅编译

⏮ 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 10 行输出

源代码 (L26)

程序输出

内存栈 (Stack Frames)

PC: 0 - 栈: []

地址	操作码	操作数	备注
(0)	INT	0 9	分配 9 个栈槽
(1)	LIT	0 5	压入常数 5
(2)	STO	0 3	存入变量 [0,3]
(3)	LIT	0 3	压入常数 3
(4)	STO	0 4	存入变量 [0,4]
(5)	LIT	0 1	压入常数 1
(6)	LIT	0 2	压入常数 2
(7)	LIT	0 3	压入常数 3
(8)	LIT	0 4	压入常数 4
(9)	LIT	0 5	压入常数 5
(10)	OPR	5 201	OPR.SET_PC: 构造集合
(11)	STO	0 5	存入变量 [0,5]
(12)	LIT	0 3	压入常数 3
(13)	LIT	0 4	压入常数 4

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	a	2	9
3	;	;	2	10
4	int	int	3	5
5	ident	b	3	9
6	;	;	3	10
7	set	set	4	5
8	ident	s1	4	9
9	;	;	4	11
10	set	set	5	5
11	ident	s2	5	9
12	;	;	5	11
13	set	set	6	5

test4.l26 — 最小算术测试

```
{
    int x;
    x = 1;
    x = 1 + 2;
    x = (1 + 2) * 3;
}
```

运行结果:

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

🔍 仅编译

⏮ 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 0 行输出

源代码 (L26)

程序输出

内存栈 (Stack Frames)

PC: 0 - 栈: []

地址	操作码	操作数	备注
(0)	INT	0 4	分配 4 个栈槽
(2)	STO	0 3	存入变量 [0,3]
(3)	LIT	0 1	压入常数 1
(4)	LIT	0 2	压入常数 2
(5)	OPR	0 2	OPR.ADD: 加
(6)	STO	0 3	存入变量 [0,3]
(7)	LIT	0 1	压入常数 1
(8)	LIT	0 2	压入常数 2
(9)	OPR	0 2	OPR.ADD: 加
(10)	LIT	0 3	压入常数 3
(11)	OPR	0 4	OPR.MUL: 乘
(12)	STO	0 3	存入变量 [0,3]
(13)	OPR	0 0	OPR.RET: 返回
(14)	OPR	0 0	OPR.RET: 返回

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	x	2	9
3	;	;	2	10
4	ident	x	3	5
5	=	=	3	7
6	number	1	3	9
7	;	;	3	10
8	ident	x	4	5
9	=	=	4	7
10	number	1	4	9
11	+	+	4	11
12	number	2	4	13
13	;	;	4	14

test5.l26 — 最小布尔测试

```
{
    bool b;
    b = true;
    b = false;
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

⏻ 仅编译

⏮ 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 0 行输出

源代码 (L26)

程序输出

内存栈 (Stack Frames)

PC: - 栈: []

地址	操作码	操作数	备注
(0)	INT	0 4	分配 4 个栈槽
(1)	LIT	0 1	压入常量 1
(2)	STO	0 3	存入变量 [0,3]
(3)	LIT	0 0	压入常量 0
(4)	STO	0 3	存入变量 [0,3]
(5)	OPR	0 0	OPR_RET: 返回
(6)	OPR	0 0	OPR_RET: 返回

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	bool	bool	2	5
2	ident	b	2	10
3	;	;	2	11
4	ident	b	3	5
5	=	=	3	7
6	true	true	3	9
7	;	;	3	13
8	ident	b	4	5
9	=	=	4	7
10	false	false	4	9
11	;	;	4	14
12	}	}	5	1

test6.l26 — 最小关系测试

```
{
    int x;
    bool b;
    b = x > 0;
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

⏻ 仅编译

⏮ 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 0 行输出

源代码 (L26)

程序输出

内存栈 (Stack Frames)

PC: - 栈: []

地址	操作码	操作数	备注
(0)	INT	0 5	分配 5 个栈槽
(1)	LOD	0 3	加载变量 [0,3]
(2)	LIT	0 0	压入常量 0
(3)	OPR	0 12	OPR_GT: >
(4)	STO	0 4	存入变量 [0,4]
(5)	OPR	0 0	OPR_RET: 返回
(6)	OPR	0 0	OPR_RET: 返回

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	x	2	9
3	;	;	2	10
4	bool	bool	3	5
5	ident	b	3	10
6	;	;	3	11
7	ident	b	4	5
8	=	=	4	7
9	ident	x	4	9
10	>	>	4	11
11	number	0	4	13
12	;	;	4	14
13	}	}	5	1

test7.l26 — 集合字面量与 in

```
{
    set s;
    int x;
    bool b;

    x = 1 + 2;
    s = {1, 2, 3};
    b = true;
    b = x > 0;
    b = x in s;
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

🔍 仅编译

🏠 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 0 行输出

● 源代码 (L26)

```
{
    set s;
    int x;
    bool b;

    x = 1 + 2;
    s = {1, 2, 3};
    b = true;
    b = x > 0;
    b = x in s;
}
```

● 程序输出

(无输出)

● 内存栈 (Stack Frames)

(单步执行时显示各帧变量)

● P-Code 指令序列

PC: - 找: {}

地址	操作码	操作数	备注
(0)	INT	0 6	分配 6 个栈槽
(1)	LIT	0 1	压入常数 1
(2)	LIT	0 2	压入常数 2
(3)	OPR	0 2	OPR.ADD: 加
(4)	STO	0 4	存入变量 [0,4]
(5)	LIT	0 1	压入常数 1
(6)	LIT	0 2	压入常数 2
(7)	LIT	0 3	压入常数 3
(8)	OPR	3 201	OPR.SET_PMC: 构造集合
(9)	STO	0 3	存入变量 [0,3]
(10)	LIT	0 1	压入常数 1
(11)	STO	0 5	存入变量 [0,5]
(12)	LOO	0 4	加载变量 [0,4]
(13)	LIT	0 0	压入常数 0

● Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	set	set	2	5
2	ident	s	2	9
3	;	;	2	10
4	int	int	3	5
5	ident	x	3	9
6	;	;	3	10
7	bool	bool	4	5
8	ident	b	4	10
9	;	;	4	11
10	ident	x	6	5
11	=	=	6	7
12	number	1	6	9
13	+	+	6	11

6.1.3 加分项测试

test8.l26 — 集合相等/不等判定

```
{
    set a;
    set b;
    bool r;

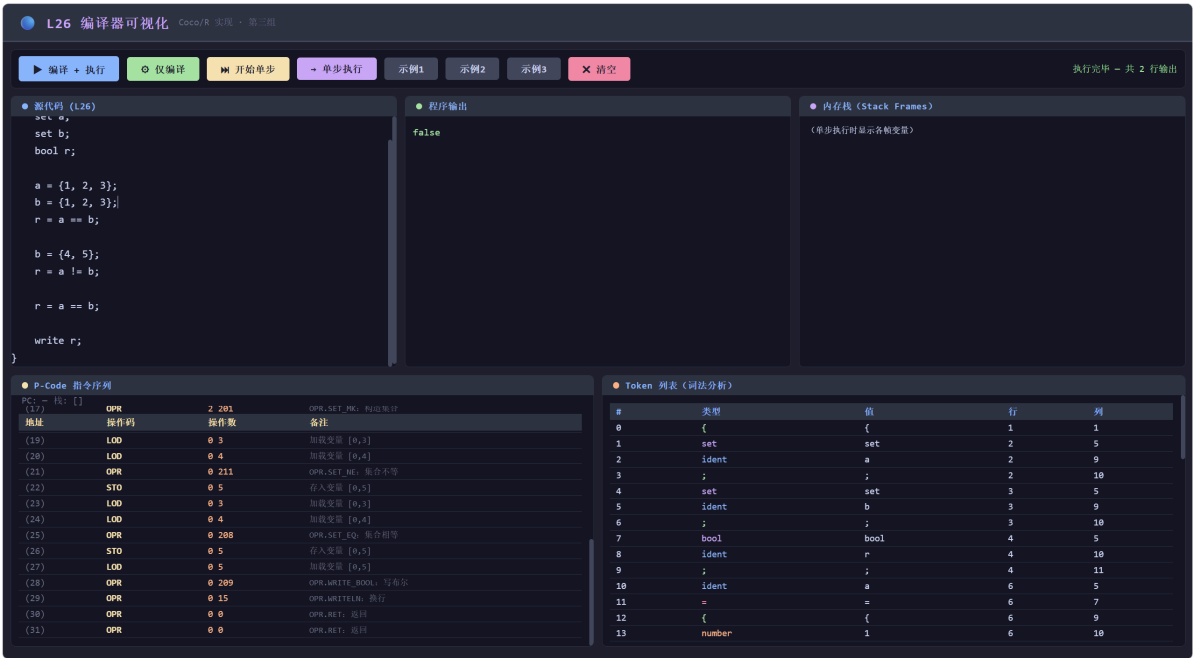
    a = {1, 2, 3};
    b = {1, 2, 3};
    r = a == b;

    b = {4, 5};
    r = a != b;

    r = a == b;

    write r;
}
```

运行结果：



test_full.l26 — 全语法综合：int/bool/set声明、赋值、union/inter/in/isempty、if条件、write输出

```

{
    int x;
    bool b;
    set s;
    set t;

    x = 10;
    s = {1, 2, 3};
    t = {4, 5};

    b = x in s;
    b = isempty(s);

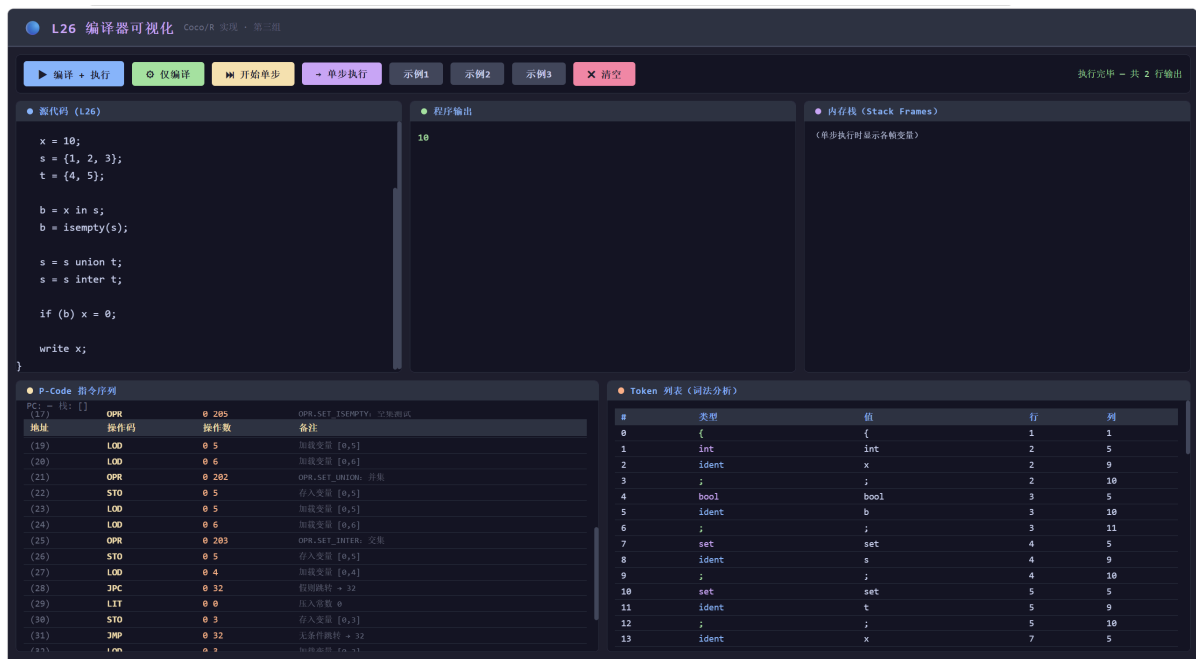
    s = s union t;
    s = s inter t;

    if (b) x = 0;

    write x;
}

```

运行结果：



6.1.4 新增扩展测试

test_arithmetic.l26 — 四则运算 + - * /、一元负号、多层括号嵌套、运算符优先级

```
{  
  
    int a;  
    int b;  
    int c;  
    int d;  
    int e;  
  
    // 基本四则运算  
    a = 10 + 5;  
    write a;  
    b = 20 - 8;  
    write b;  
    c = 6 * 7;  
    write c;  
    d = 100 / 4;  
    write d;  
  
    // 负数（一元负号）  
    e = -a;  
    write e;  
  
    // 括号嵌套  
    e = (a + b) * (c - d);  
    write e;  
  
    // 复杂表达式：运算符优先级  
    e = a + b * c - d / 2;  
    write e;  
  
    // 多层括号  
    e = ((a + 1) * (b - 2)) + ((c / 3) - d);  
    write e;
```

```
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三期

▶ 编译 + 执行

🔍 仅编译

🏠 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 16 行输出

源代码 (L26)

```
e = -a;
write e;

// 括号嵌套
e = (a + b) * (c - d);
write e;

// 复杂表达式: 运算符优先级
e = a + b * c - d / 2;
write e;

// 多层括号
e = ((a + 1) * (b - 2)) + ((c / 3) - d);
write e;
}
```

程序输出

```
15
12
42
25
-15
459
587
149
```

内存栈 (Stack Frames)

(单步执行时显示各帧变量)

P-Code 指令序列

地址	OPR	操作数	备注
(19)	LOD	0 5	加载变量 [0,5]
(20)	OPR	0 14	OPR.WRITELN: 写整数
(21)	OPR	0 15	OPR.WRITELN: 换行
(22)	LIT	0 100	压入整数 100
(23)	LIT	0 4	压入整数 4
(24)	OPR	0 5	OPR.DIV: 除
(25)	STO	0 6	存入变量 [0,6]
(26)	LOD	0 6	加载变量 [0,6]
(27)	OPR	0 14	OPR.WRITELN: 写整数
(28)	OPR	0 15	OPR.WRITELN: 换行
(29)	LOD	0 3	加载变量 [0,3]
(30)	OPR	0 1	OPR.NEG: 取负
(31)	STO	0 7	存入变量 [0,7]
(32)	OPR	0 7	OPR.WRITELN: 写整数

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	a	2	9
3	;	;	2	10
4	int	int	3	5
5	ident	b	3	9
6	;	;	3	10
7	int	int	4	5
8	ident	c	4	9
9	;	;	4	10
10	int	int	5	5
11	ident	d	5	9
12	;	;	5	10
13	int	int	6	5

test_control.l26 — if-else 双分支、嵌套 if、while 循环累加、&& || ! 布尔组合

```
{
    int x;
    int y;
    int sum;
    bool flag;
    bool done;

    x = 10;
    y = 5;

    // if-else 分支
    if (x > y) {
        write x;
    } else {
        write y;
    }

    // 复杂布尔条件: && 和 || 组合
    if ((x > 0) && (y > 0)) {
        sum = x + y;
        write sum;
    }

    // 嵌套 if
    if (x > y) {
        if (x > 0) {
            write x;
        }
    }

    // while 循环累加
```

```

sum = 0;
x = 5;
while (x > 0) {
    sum = sum + x;
    x = x - 1;
}
write sum;

// 逻辑非 ! 的使用
flag = true;
done = !flag;
write done;

// 复杂条件: !(A || B) 等价于 !A && !B
if (!(x > 10) || (y > 10))) {
    write 0;
}
}

```

运行结果:

L26 编译器可视化 Coco/R 实现 · 第三组

编译 + 执行 仅编译 开始单步 单步执行 示例1 示例2 示例3 清空

执行完毕 - 共 12 行输出

源代码 (L26)

```

sum = sum + x;
x = x - 1;
}
write sum;

// 逻辑非 ! 的使用
flag = true;
done = !flag;
write done;

// 复杂条件: !(A || B) 等价于 !A && !B
if (!(x > 10) || (y > 10))) {
    write 0;
}
}

```

程序输出

```

10
15
10
15
false
0

```

内存栈 (Stack Frames)

(单步执行时显示各帧变量)

P-Code 指令序列

地址	操作码	操作数	备注
(17)	OPR	0 15	OPR.WRITELN: 换行
(19)	LIT	0 0	压入常数 0
(20)	OPR	0 12	OPR.GT: >
(21)	LOD	0 4	加载变量 [0,4]
(22)	LIT	0 0	压入常数 0
(23)	OPR	0 12	OPR.GT: >
(24)	OPR	0 18	OPR.AND: 与
(25)	JPC	0 35	跳转跳转 + 35
(26)	INT	0 3	分配 3 个栈槽
(27)	LOD	1 3	加载变量 [1,3]
(28)	LOD	1 4	加载变量 [1,4]
(29)	OPR	0 2	OPR.ADD: 加
(30)	STO	1 5	存入变量 [1,5]
(31)	LOD	1 5	加载变量 [1,5]
(32)	OPR	0 4	OPR.LOBYTE: 加载字节

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	x	2	9
3	;	;	2	10
4	int	int	3	5
5	ident	y	3	9
6	;	;	3	10
7	int	int	4	5
8	ident	sum	4	9
9	;	;	4	12
10	bool	bool	5	5
11	ident	flag	5	10
12	;	;	5	14
13	bool	bool	6	5

test_io.l26 — write int/bool/set 三种类型、read 读取整数、运算及集合 union 后 write

```

{
    int a;
    int b;
    bool flag;
    set s;
    set t;

    // write int
    a = 42;
    write a;

    // write bool
    flag = true;
    write flag;
}

```

```
flag = false;
write flag;

// write set
s = {1, 2, 3};
write s;

// read int
read a;
write a;

// read + 运算后 write
read b;
a = a + b;
write a;

// write 集合运算结果
t = {4, 5};
s = s union t;
write s;

// write 比较结果
flag = a > b;
write flag;
}
```

运行结果：

L26 编译器可视化Coco/R 实现 · 第三组

▶ 编译 + 执行

🔍 仅编译

🏠 开始单步

➡ 单步执行

示例1

示例2

示例3

✖ 清空

执行完毕 - 共 16 行输出

源代码 (L26)

```
// read + 运算后 write
read b;
a = a + b;
write a;

// write 集合运算结果
t = {4, 5};
s = s union t;
write s;

// write 比较结果
flag = a > b;
write flag;
}
```

程序输出

```
42

true

false

{1, 2, 3}

1

3

{1, 2, 3, 4, 5}
```

内存栈 (Stack Frames)

(单步执行时显示当前变量)

P-Code 指令序列

PC: 1	找: []		
(17)	LIT	0 2	压入常量 2
(19)	OPR	3 201	OPR.SET_PRG: 构造集合
(20)	STO	0 6	存入变量 [0,6]
(21)	LOD	0 6	加载变量 [0,6]
(22)	OPR	0 210	OPR.WRITE_SET: 写集合
(23)	OPR	0 15	OPR.WRITELN: 换行
(24)	OPR	0 16	OPR.READ: 读输入
(25)	STO	0 3	存入变量 [0,3]
(26)	LOD	0 3	加载变量 [0,3]
(27)	OPR	0 14	OPR.WRITE: 写结果
(28)	OPR	0 15	OPR.WRITELN: 换行
(29)	OPR	0 16	OPR.READ: 读输入
(30)	STO	0 4	存入变量 [0,4]
(31)	LOD	0 3	加载变量 [0,3]
(32)	OPR	0 4	OPR.WRITE: 写结果

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	a	2	9
3	;	;	2	10
4	int	b	3	5
5	ident	b	3	9
6	;	;	3	10
7	bool	bool	4	5
8	ident	flag	4	10
9	;	;	4	14
10	set	set	5	5
11	ident	=	5	9
12	;	;	5	10
13	set	set	6	5

test_set_advanced.l26 — 空集 {}、链式 union/inter、add/remove 构建与清空、in 成员测试、isempty、while 中 add

```
{
    set empty;
    set s1;
    set s2;
    set s3;
    int elem;
```

```
bool found;
bool allCleared;

// 空集合
empty = {};
write empty;

// 初始集合
s1 = {10, 20, 30};
s2 = {30, 40, 50};

// 三重 union
s3 = s1 union s2;
write s3;

// union 后 inter
s3 = s3 inter s2;
write s3;

// 链式: 集合间连续运算
s1 = {1, 2, 3};
s2 = {3, 4, 5};
s3 = s1 union s2;
s3 = s3 inter s1;
write s3;

// add 逐个构建集合
s1 = {};
add s1 100;
add s1 200;
add s1 300;
write s1;

// remove 逐个清空集合
remove s1 100;
remove s1 200;
remove s1 300;
write s1;

// in 成员测试
s1 = {1, 2, 3, 4, 5};
found = 3 in s1;
write found;
found = 99 in s1;
write found;

// isempty 测试
allCleared = isempty(empty);
write allCleared;
allCleared = isempty(s1);
write allCleared;

// while 中用 add 构建集合
elem = 5;
s1 = {};
while (elem > 0) {
```

```

    add s1 elem;
    elem = elem - 1;
}
write s1;
}

```

运行结果：

The screenshot shows the L26 compiler visualization interface. The top bar includes buttons for '编译 + 执行' (Compile + Run), '仅编译' (Only Compile), '开始单步' (Start Step-by-Step), '单步执行' (Step-by-Step Execution), and '清空' (Clear). The main area is divided into three panels:

- 源代码 (L26)**: Contains the source code for the L26 program, including comments and assembly-like instructions.
- 程序输出**: Displays the output of the program, showing a sequence of values: {}, {}, true, false, true, false, {}.
- 内存栈 (Stack Frames)**: Shows the current stack frames, indicating the execution state.

Below the main panels, there are two additional sections:

- P-Code 指令序列**: A table showing the sequence of P-Code instructions, including their addresses, operations, operands, and comments.
- Token 列表 (词法分析)**: A table showing the tokens generated by the lexer, including their indices, types, values, and line/column positions.

6.2 错误处理测试

6.2.1 未声明变量 (test_error_undeclared.l26)

共4处错误，覆盖赋值右值/运算操作数/条件表达式/write 参数中使用未声明变量四种场景。

```

{
    int x;

    x = 10;          // OK: x 已声明

    // 错误1: 赋值时使用未声明变量
    x = y;           // ERROR: y 未声明

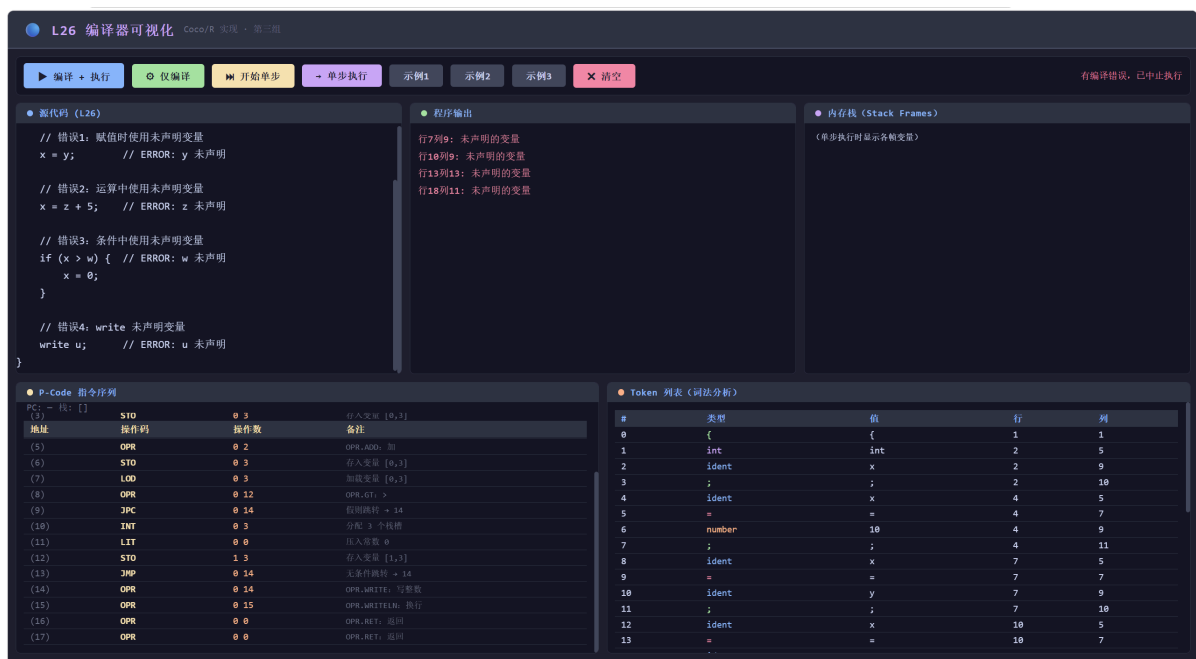
    // 错误2: 运算中使用未声明变量
    x = z + 5;       // ERROR: z 未声明

    // 错误3: 条件中使用未声明变量
    if (x > w) {     // ERROR: w 未声明
        x = 0;
    }

    // 错误4: write 未声明变量
    write u;         // ERROR: u 未声明
}

```

运行结果：

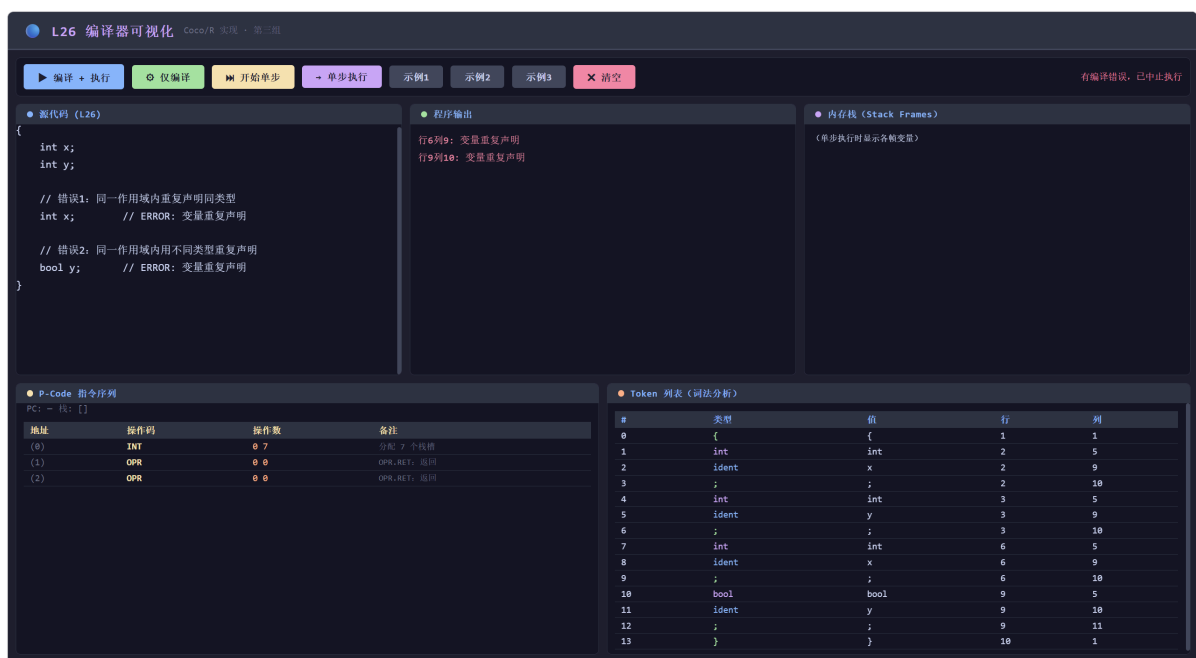


6.2.2 重复声明 (test_error_duplicate.l26)

共2处错误，覆盖同类型重复声明和不同类型重复声明。

```
{  
    int x;  
    int y;  
  
    // 错误1: 同一作用域内重复声明同类型  
    int x;          // ERROR: 变量重复声明  
  
    // 错误2: 同一作用域内用不同类型重复声明  
    bool y;         // ERROR: 变量重复声明  
}
```

运行结果:



6.2.3 类型不匹配 (test_error_type.l26)

共5处错误，覆盖 int=bool、bool=int、bool=set、set=bool、int=set 五种赋值类型不匹配场景。

```
{
    int a;
    bool b;
    set s;

    a = 10;      // OK: int = int
    b = true;    // OK: bool = bool
    s = {1, 2};  // OK: set = set

    // 错误1: int 变量赋 bool 值
    a = true;    // ERROR: 赋值类型不匹配

    // 错误2: bool 变量赋 int 值
    b = 42;      // ERROR: 赋值类型不匹配

    // 错误3: bool 变量赋 set 值
    b = {};      // ERROR: 赋值类型不匹配

    // 错误4: set 变量赋 bool 值
    s = false;   // ERROR: 赋值类型不匹配

    // 错误5: int 变量赋 set 值
    a = {1, 2};  // ERROR: 赋值类型不匹配
}
```

运行结果：

L26 编译器可视化

编译 + 执行

仅编译

开始单步

单步执行

示例1

示例2

示例3

清空

有编译错误，已中止执行

源代码 (L26)

```
// 错误1: int 变量赋 bool 值
a = true;    // ERROR: 赋值类型不匹配

// 错误2: bool 变量赋 int 值
b = 42;      // ERROR: 赋值类型不匹配

// 错误3: bool 变量赋 set 值
b = {};      // ERROR: 赋值类型不匹配

// 错误4: set 变量赋 bool 值
s = false;   // ERROR: 赋值类型不匹配

// 错误5: int 变量赋 set 值
a = {1, 2};  // ERROR: 赋值类型不匹配
}
```

程序输出

```
行11到9: 赋值类型不匹配
行14到9: 赋值类型不匹配
行17到18: 赋值类型不匹配
行28到9: 赋值类型不匹配
行23到14: 赋值类型不匹配
```

内存栈 (Stack Frames)

(单步执行时显示当前变量)

P-Code 指令序列

PC: 0 行: 1

地址	操作码	操作数	备注
(0)	INT	0 6	分配 6 个栈帧
(1)	LIT	0 10	压入常量 10
(2)	STO	0 3	存入变量 [0,3]
(3)	LIT	0 1	压入常量 1
(4)	STO	0 4	存入变量 [0,4]
(5)	LIT	0 1	压入常量 1
(6)	LIT	0 2	压入常量 2
(7)	OPR	2 201	OPR.SET PC: 构造集合
(8)	STO	0 5	存入变量 [0,5]
(9)	LIT	0 1	压入常量 1
(10)	STO	0 3	存入变量 [0,3]
(11)	LIT	0 42	压入常量 42
(12)	STO	0 4	存入变量 [0,4]
(13)	OPR	0 201	OPR.SET PC: 构造集合

Token 列表 (词法分析)

#	类型	值	行	列
0	{	{	1	1
1	int	int	2	5
2	ident	a	2	9
3	;	;	2	10
4	bool	bool	3	5
5	ident	b	3	10
6	;	;	3	11
7	set	set	4	5
8	ident	s	4	9
9	;	;	4	10
10	ident	a	6	5
11	=	=	6	7
12	number	10	6	9
13	;	;	6	11

6.2.4 集合操作类型错误 (test_error_set_ops.l26)

共7处错误:

语句	错误原因
<code>add n 5</code>	add 目标 n 是 int, 必须是 set
<code>add b 5</code>	add 目标 b 是 bool, 必须是 set
<code>remove n 3</code>	remove 目标 n 是 int, 必须是 set
<code>s union n</code>	union 右侧 n 是 int, 必须是 set
<code>s inter b</code>	inter 右侧 b 是 bool, 必须是 set
<code>n in n</code>	in 右侧 n 是 int, 必须是 set
<code>isempty(n)</code>	isempty 参数 n 是 int, 必须是 set

```
{
    int n;
    bool b;
    set s;
    set t;

    n = 10;
    b = true;
    s = {1, 2};
    t = {3, 4};

    // 错误1: add 到非 set 变量
    add n 5;           // ERROR: add 目标必须是 set 类型

    // 错误2: add 到 bool 变量
    add b 5;           // ERROR: add 目标必须是 set 类型

    // 错误3: remove 从非 set 变量
    remove n 3;        // ERROR: remove 目标必须是 set 类型

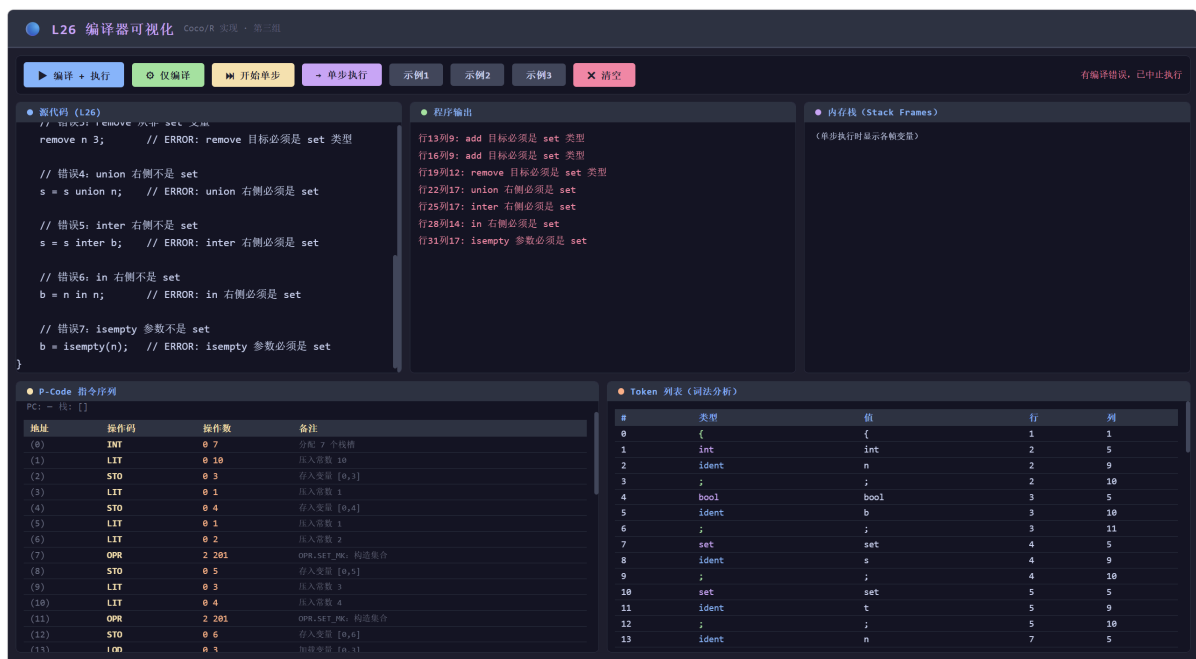
    // 错误4: union 右侧不是 set
    s = s union n;      // ERROR: union 右侧必须是 set

    // 错误5: inter 右侧不是 set
    s = s inter b;      // ERROR: inter 右侧必须是 set

    // 错误6: in 右侧不是 set
    b = n in n;         // ERROR: in 右侧必须是 set

    // 错误7: isempty 参数不是 set
    b = isempty(n);     // ERROR: isempty 参数必须是 set
}
```

运行结果:



6.2.5 集合非法关系运算符 (test_error_set_relop.l26)

共4处错误，覆盖集合使用 `<` `>` `<=` `>=` 四种非法关系运算符。集合只支持 `==` 和 `!=`。预期错误信息："集合只支持 == 与 !="。

```
{
    set s;
    set t;
    bool b;

    s = {1, 2, 3};
    t = {4, 5, 6};

    // 错误1: 集合使用 < 比较
    b = s < t;          // ERROR: 集合只支持 == 与 !=

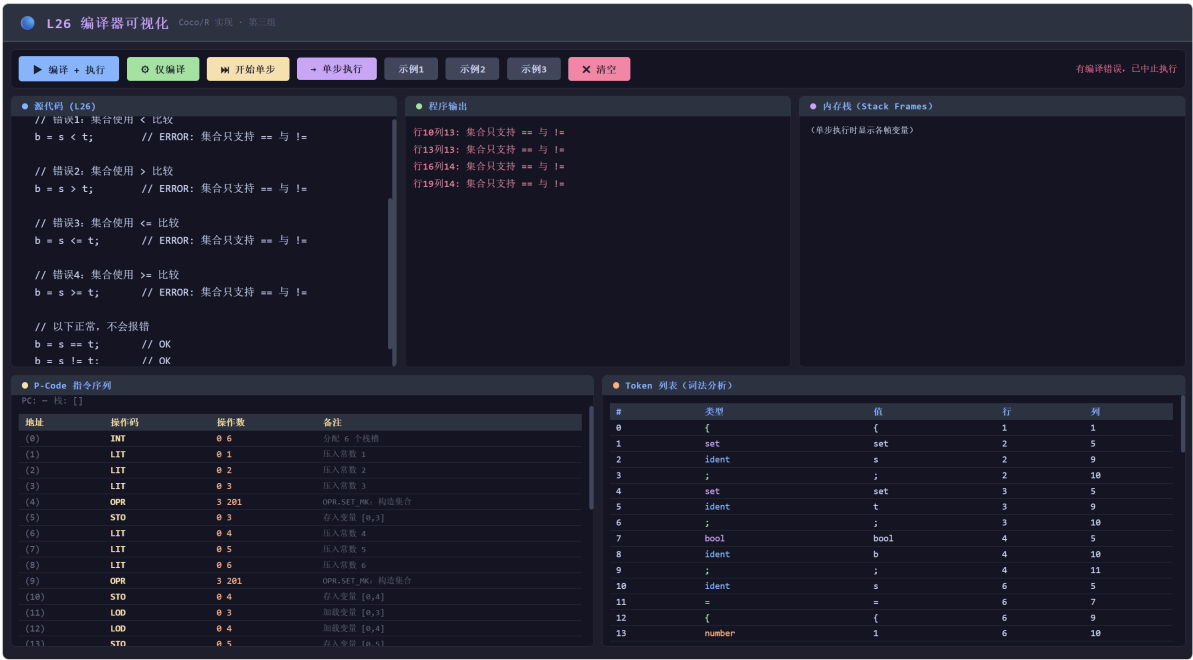
    // 错误2: 集合使用 > 比较
    b = s > t;          // ERROR: 集合只支持 == 与 !=

    // 错误3: 集合使用 <= 比较
    b = s <= t;         // ERROR: 集合只支持 == 与 !=

    // 错误4: 集合使用 >= 比较
    b = s >= t;         // ERROR: 集合只支持 == 与 !=

    // 以下正常，不会报错
    b = s == t;         // OK
    b = s != t;         // OK
}
```

运行结果：



7 前端可视化与单步调试

7.1 界面布局

源代码	程序输出	内存栈(Frames)
P-Code指令序列(50%)		Token列表(50%)

7.2 单步调试功能

- 开始单步：编译源码，初始化VM，准备逐条执行
- 单步执行：每次执行一条P-Code指令，高亮当前PC行，实时更新内存栈帧
- 内存栈展示：沿静态链遍历所有帧，从外到内层级显示SL/DL/RA/局部变量
- 错误命中：单步至对应源码行时，输出区显示编译错误信息（同一错误只显示一次）
- read支持：执行到read语句时弹出输入对话框

7.3 前后端通信

前端fetch POST源码到 `localhost:8765/compile` → Node.js server写临时文件 → 调用 `l26compiler.exe --json` → 返回JSON (tokens + pcode + errors) → 前端渲染并执行

8 加分项说明与AI使用说明

8.1 已实现加分功能

加分项	实现说明
界面显示P-Code+内存管理+单步运行	前端index.html完整支持，含JS VM、逐条单步、栈帧可视化

加分项	实现说明
集合相等/不等判定	SET_EQ(208)/SET_NE(211), 编译器根据操作数类型自动选择

8.2 AI使用说明

开发过程中使用了 WorkBuddy 和 Claude Code 作为辅助工具。AI 的主要作用为代码生成优化和调试辅助——帮助快速生成框架性代码、排查语法错误、撰写文档草稿。所有 AI 输出的代码均经过我逐行审阅、理解、修改和验证，核心设计决策和语义逻辑由我独立完成。

编译器核心模块：

整体架构和核心逻辑由我自己完成。AI 主要发挥了以下辅助作用：

- **L26.atg 文法文件**：语法规则本身由我独立设计完成。AI 辅助了 Coco/R ATG 格式语法的适配，比如将非标准 pragma 写法迁移到标准语法、解决编译时的格式报错。
- **PCode.h (P-Code 指令定义与代码生成器)**：PL/0 标准 8 条指令 (LIT/OPR/LOD/STO/CAL/INT/JMP/JPC) 16 个 OPR 子码和扩展的集合操作子码由我自己根据课件定义。AI 辅助了 C++ 结构体封装、跳转回填等的实现细节。
- **VM.h (虚拟机)**：指令执行循环和栈帧管理由我对标 PL/0 标准实现。AI 辅助了具体写法优化、边界条件检查等的补充。
- **SymTab.h (符号表)**：符号表结构、作用域入栈出栈、变量查找逻辑由我自己设计。AI 辅助了代码组织结构的优化和模板化封装。
- **main.cpp**：双模式架构设计、整体流程控制由我完成，AI 辅助了 `--json` 输出模式用于与前端对接的实现。

前端可视化界面：

`Frontend/index.html` 和 `Frontend/server.js` 的整体代码由 AI 生成，但并非 AI"自主决定"写什么。具体流程是：

1. 由我决定前端需要三个核心功能：P-Code 指令列表展示、运行时栈/内存可视化、逐步执行调试；
2. 我指定技术方案：纯 HTML + JS 实现前端虚拟机、Node.js 做桥接服务调用后端 exe；
3. 我要求前端 JS 虚拟机的指令执行逻辑必须与后端 C++ VM.h 逐条对齐，确保前端展示的运行状态与后端一致；
4. AI 生成代码后，我逐段审阅、测试，修正了集合类型在前端渲染时的显示 bug，调整了界面布局使其更直观。

测试用例：

- 我自己先编写了几个基础测试用例；
- 在这些用例通过后，我让 AI 基于我的用例风格和 L26 语言规范，补充了更完善的测试用例，包括边界条件（空集合、嵌套作用域变量遮蔽）、组合场景（布尔+集合混合表达式）、各类语义错误检测（未声明变量、类型不匹配、重复声明）等。

使用总结

AI 主要承担了前端展示层和文档辅助工作。编译器的核心技术部分均由人工主导完成。AI 在调试冲突、发现 token 编号错误、生成测试边界用例等方面起到了有效的辅助作用，但所有设计决策均经我确认。这次在编译原理大作业的开发过程使用ai，让我对"怎么和 AI 协作"有了不少体会：

1. **AI 能加速实现，但不能替代理解。** 最典型的是 P-Code 虚拟机。AI 可以五分钟写出一个能跑的 VM 框架，但如果我不理解 PL/0 的栈帧布局和静态链机制，出了 bug 根本无从查起。AI 帮我省掉的是敲键盘的时间，不是啃课件的时间。
2. **需求描述越精确，AI 产出越靠谱。** 前期我给 AI 的指令比较模糊（比如"帮我写个前端"），出来的东西偏离很大。后来我学会了先把功能拆解清楚、指定技术路线（纯 HTML + JS、不需框架）、甚至给出数据结构协议（前后端 JSON 字段名和格式），AI 的代码质量立刻提升很多。
3. **AI 会犯错。** 有几次 AI 生成的代码在编译时报错，我丢回去让它修，它又生成一段同样有问题的代码，来回好几次。最后是我自己看报错信息、对比 ExpressionProject 模板修好的。
4. **对"AI 辅助开发"有了边界感。** 哪些事可以放心交给 AI（框架性代码、重复性语义动作、文档初稿），哪些事必须自己把控（文法设计、核心算法、类型系统规则、最终验证），这次实践帮我逐渐建立了一套判断标准。这对以后在更大的工程里使用 AI 会很有帮助。